

/*

----- French -----

Exercice 1:

- 1) Ecrire une fonction Allouer (int N), qui permet d'allouer un espace pour une séquence ADN de longueur N spécifiée par l'utilisateur.
 - 2) Ecrire une fonction ADN_Sequences (int N), qui génère une séquence ADN aléatoire de longueur N.
 - 3) Ecrire la fonction Affiche, qui permet d'afficher la séquence ADN de longueur N.
 - 4) Ecrire la fonction Compte_Nucléotides, qui à partir de la séquence générée, permet de compter le nombre de Cystéines (C), Adénine (A), Thymine (T) et guanine (G).
 - 5) Écrire une fonction Complement qui étant donné une séquence ADN, renvoie son complémentaire. Chaque base possède une base complémentaire avec laquelle elle peut s'associer : "A" et "T" sont complémentaires, et "C" et "G" sont complémentaires.
- Exemple
- Séquence d'ADN
- ATCGTTTAGCTATATTTGCCCTTCAACCGAAACCAACTGCAGTTC
- Brin complémentaire
- TAGCAAATCGATATAAACGGGAAGTTGGCTTTGGTTGACGTCAAG
- 6) Ecrire la fonction Codons qui permet d'extraire les codons (triplets) d'une séquence ADN.
- Exemple
- Séquence d'ADN
- ATCGTTTAGCTATATTTGCCCTTCAACCGAAACCAACTGCAGTTC
- Les codons
- ATC GTT TAG CTA TAT TTG CCC TTC AAC CGA AAC CAA CTG CAG TTC
- 7) Ecrire la fonction codon_start(ADN), qui prenant en entrée une séquence d'ADN renvoie les positions de tous les codons start 'ATG' trouvés (on indiquera l'indice du 'A'). Traiter le cas où aucun codon 'ATG' n'est trouvé.
 - 8) Écrire une fonction codon_stop (ADN, i) prenant en entrée une séquence d'ADN (chaîne de caractères) et l'indice i du début d'un codon start et renvoyant l'indice du début du premier codon stop (de la forme TAA, TAG ou TGA) rencontré.
 - 9) Ecrire la fonction SupprimeDuplications qui permet de supprimer les éléments en multiples dans la séquence, et renvoyer la nouvelle la séquence. Gérer le problème de l'espace non alloué.
 - 10) Ecrire un programme principal de test.

*/

/*

----- English -----

Exercise 1:

- 1) Write a function Allocate(int N), which allows to allocate space for a DNA sequence of length N specified by the user.
- 2) Write a function ADN_Sequences (int N), which generates a random DNA sequence of length N.
- 3) Write the Display function, which displays the DNA sequence of length N.
- 4) Write the Count_Nucleotides function, which from the generated sequence, allows to count the number of Cysteines (C), Adenine (A), Thymine (T) and Guanine (G).
- 5) Write a Complement function which, given a DNA sequence, returns its complement. Each base has a complementary base with which it can associate: "A" and "T" are complementary, and "C" and "G" are complementary.

Example

DNA sequence

ATCGTTTAGCTATATTTGCCCTTCAACCGAAACCAACTGCAGTTC

Complementary strand

TAGCAAATCGATATAAACGGGAAGTTGGCTTTGGTTGACGTCAAG

- 6) Write the Codons function which allows you to extract the codons (triplets) from a DNA sequence.

Example

DNA sequence

ATCGTTTAGCTATATTTGCCCTTCAACCGAAACCAACTGCAGTTC

codons

ATC GTT TAG CTA TAT TTG CCC TTC AAC CGA AAC CAA CTG CAG TTC

- 7) Write the codon_start(DNA) function, which taking a DNA sequence as input returns the positions of all the 'ATG' start codons found (we will indicate the index of the 'A'). Treat the case where no 'ATG' codon is found.
- 8) Write a function codon_stop (DNA, i) taking as input a DNA sequence (character string) and the index i of the start of a start codon and returning the index of the start of the first stop codon (of the TAA, TAG or TGA form) encountered.
- 9) Write the SuppressDuplications function which allows multiple elements to be suppressed in the sequence, and return the new sequence. Manage the problem of unallocated space.
- 10) Write a main test program.

*/

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

/* Function in use */
char *allocate(int length);
char *ADN_sequence(char *DNA, int length);
void display(char *DNA, int length);
void Count_Nucleotides(char *DNA, int length);
char *complement(char *DNA, int length);
void codons(char *DNA, int *length);
void codon_start(char *DNA, int length);
void codon_stop(char *DNA, int length);
void remove_duplications(char *DNA, int length);

/* Function make way clear*/
int get_length(int length);

int main()
{
    int length = get_length(length);
    char *DNA;
    DNA = allocate(length);
    DNA = ADN_sequence(DNA, length);
    printf("\nThe generated sequence:_____ \n");
    display(DNA, length);

    Count_Nucleotides(DNA, length);
    printf("\nComplement the generated sequence:_____A becomes T__T becomes A_____C becomes G__G
becomes C\n");
    display(DNA, length);
    DNA = complement(DNA, length);
    display(DNA, length);

    codons(DNA, &length);
    codon_start(DNA, length);
    codon_stop(DNA, length);
}

```

```
remove_duplications(DNA, length);
```

```
printf("\n");
```

```
return 0;
```

```
}
```

```
char *allocate(int length)
```

```
{
```

```
    char *DNA;
```

```
    DNA = (char *)malloc(length * sizeof(char));
```

```
    return DNA;
```

```
}
```

```
char *ADN_sequence(char *DNA, int length)
```

```
{
```

```
    int i, rand_x;
```

```
    char ADN_letters[4] = {'A', 'T', 'C', 'G'};
```

```
    for (i = 0; i < length; i++)
```

```
    {
```

```
        rand_x = rand() % 4;
```

```
        DNA[i] = ADN_letters[rand_x];
```

```
    }
```

```
    return DNA;
```

```
}
```

```
void display(char *DNA, int length)
```

```
{
```

```
    printf("\t\t");
```

```
    for (int i = 0; i < length; i++)
```

```
        printf("%c", *(DNA + i));
```

```
    printf("\n");
```

```
}
```

```
void Count_Nucleotides(char *DNA, int length)
```

```
{
```

```
    int i, nA = 0, nT = 0, nC = 0, nG = 0;
```

```
    for (i = 0; i < length; i++)
```

```
    {
```

```

    if ((*DNA + i) == 'A'))
        nA++;
    if ((*DNA + i) == 'T'))
        nT++;
    if ((*DNA + i) == 'C'))
        nC++;
    if ((*DNA + i) == 'G'))
        nG++;
}

printf("\nAfter calculating Cysteines (C), Adenine (A), Thymine (T) and Guanine (G):\n");
printf("\t\tNumber of Adenine,\t (A) = %d\n", nA);
printf("\t\tNumber of Thymine,\t (T) = %d\n", nT);
printf("\t\tNumber of Cysteines,\t (C) = %d\n", nC);
printf("\t\tNumber of Guanine,\t (G) = %d\n", nG);
}

```

```

char *complement(char *DNA, int N)

```

```

{
    int i;
    for (i = 0; i < N; i++)
    {
        if ((*DNA + i) == 'A')
        {
            *(DNA + i) = 'T';
            continue;
        }
        if ((*DNA + i) == 'T')
        {
            *(DNA + i) = 'A';
            continue;
        }
        if ((*DNA + i) == 'C')
        {
            *(DNA + i) = 'G';
            continue;
        }
        if ((*DNA + i) == 'G')
        {

```

```

        *(DNA + i) = 'C';
        continue;
    }
}
return DNA;
}

```

```

void codons(char *DNA, int *length)
{
    int i;
    printf("\nAfter seperating codons:_____ \n\t\t");
    for (i = 0; i < (*length); i += 3)
    {
        printf("%c%c%c", *(DNA + i), *(DNA + i + 1), *(DNA + i + 2));
        printf(" ");
    }
    printf("\n");
}

```

```

void codon_start(char *DNA, int length)
{
    int i, j, ATG;
    printf("\nSearching on ATG codon:_____ \n");
    for (i = 0; i < length; i++)
    {
        if ((DNA[i] == 'A') && (DNA[i + 1] == 'T') && (DNA[i + 2] == 'G'))
            printf("\t\tCodon ATG found at %d.\n", i + 1);
        else
            continue;
    }
}

```

```

void codon_stop(char *DNA, int length)
{
    int i, j, ATG;
    printf("\nSearching on TAA, TAG & TGA codons:_____ \n");
    for (i = 0; i < length; i++)
    {

```

```

if ((DNA[i] == 'T') && (DNA[i + 1] == 'A') && (DNA[i + 2] == 'A'))
    printf("\t\tCodon TAA found at %d.\n", i + 1);
if ((DNA[i] == 'T') && (DNA[i + 1] == 'A') && (DNA[i + 2] == 'G'))
    printf("\t\tCodon TAG found at %d.\n", i + 1);
if ((DNA[i] == 'T') && (DNA[i + 1] == 'G') && (DNA[i + 2] == 'A'))
    printf("\t\tCodon TGA found at %d.\n", i + 1);
else
    continue;

```

```

}

```

```

void remove_duplications(char *DNA, int length)

```

```

{

```

```

    int i, j, k;

```

```

    for (i = 0; i < length - 1; i++)

```

```

    {

```

```

        for (j = i + 1; j < length; j++)

```

```

        {

```

```

            if ((*DNA + i) == (*DNA + j))

```

```

            {

```

```

                for (k = j; k < length; k++)

```

```

                {

```

```

                    if (k < j)

```

```

                        *(DNA + k) = *(DNA + j);

```

```

                    if (k >= j)

```

```

                        *(DNA + k) = *(DNA + k + 1);

```

```

                }

```

```

                length--;

```

```

            }

```

```

        else

```

```

            j++;

```

```

        }

```

```

    }

```

```

    printf("\nAfter removing the duplicate letters from the codon:_____\n");

```

```

    display(DNA, length);

```

```

}

```

```

////////////////////////////////////

```

```
int get_length(int length)
{
    printf("Enter number of squences: ");
    scanf("%d", &length);
    return length;
}
```

```
/*
```

Edit by: MOHAMMED ELIDRISSI LAOUKILI
#idriCoding | #idriLogic | #idri
2021/2022

لا اله الا الله

```
*/
```